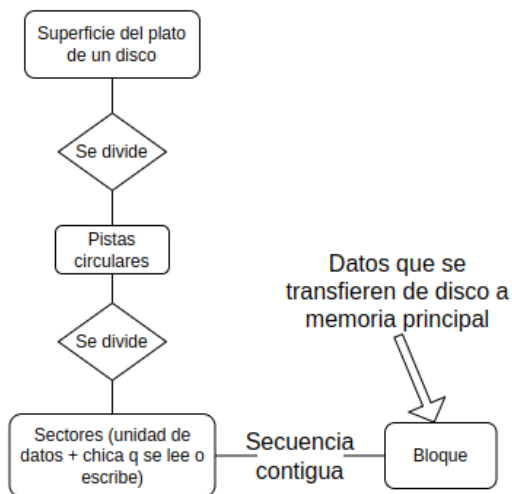


Almacenamiento de tablas en archivos e índices



- ¿Cómo almacenar una tabla de BD relacional en un archivo? => usar enfoques de organización de archivos.
- ¿Cómo acceder eficientemente a registros de una tabla? => usar archivo de índice

Organización de registros en archivos

- Con respecto a la ubicación de las tablas de la BD tenemos como alternativas:
 1. Cada tabla se aloja en archivo separado.
 2. Registros de diferentes tablas se alojan en el mismo archivo (agrupamiento de tablas en archivos).

Con respecto a la ubicación de los registros en un archivo:

1. Heap: un registro puede almacenarse en cualquier lugar del archivo donde hay espacio.
2. Secuencial: almacenar registros de una tabla en orden secuencial con respecto al valor de una clave de búsqueda de cada registro.

Organización de archivos secuencial

La organización secuencial de archivos es adecuada para procesar datos en orden, con registros enlazados y almacenados según una clave de búsqueda (que es 1 o + atributos) para optimizar accesos. Los registros se almacenan físicamente cerca del orden de la clave para minimizar accesos a bloques. Las eliminaciones usan punteros, y las para inserciones se localiza la posición adecuada y se inserta, o se usa un bloque de overflow. Requiere reorganización periódica, pero es poco eficiente para consultas que combinan varias tablas.

Agrupamiento de tablas en archivos

- Varias tablas van en un archivo
- Resulta en registros de tamaño variable.
- Se pueden agregar cadenas de punteros para enlazar registros de una relación particular.
- Suposiciones:

Cada tabla se almacena en un archivo y cada archivo se usa para una tabla.

Un archivo es una secuencia de registros y un registro es una secuencia de campos.

El tamaño de registro es fijo

Algunos conceptos

- Un archivo de BD es particionado en bloques de tamaño fijo.
- Un búfer en memoria principal es usado para almacenar copias de bloques de disco.
- Meta del SGBD: Minimizar cantidad de transferencias de bloques entre disco y memoria.

Índices

- Los archivos de índices permiten acceder más rápido a los datos deseados de una tabla, que corresponden a ciertos atributos específicos. Cada entrada es <clave de búsqueda, puntero hacia el registro correspondiente>. Estos archivos requieren almacenamiento en disco y deben actualizarse, ocupan menos que el archivo original de la tabla.

- Formas de un índice ordenado:

- o Una lista de entradas ordenada por valor de clave de búsqueda

- o Un árbol ordenado: además de otras propiedades para el ordenamiento, en cada nodo del árbol las entradas están ordenadas por valor de clave de búsqueda.

- Los índices se clasifican en:

- Sea f un archivo secuencialmente ordenado....

- Índice primario: índice cuya clave de búsqueda especifica el orden secuencial del archivo f.

- Índice secundario: índice cuya clave de búsqueda especifica un orden diferente del orden secuencial del archivo. Cada registro del índice apunta a un bucket que contiene punteros a todos los registros actuales con el valor particular de clave de búsqueda. Los índices secundarios deben ser densos.

- Índice denso: hay una entrada del índice por cada valor de la clave de búsqueda en el archivo.

- Índice disperso: contiene registros de índice para solo algunos valores de clave de búsqueda. Se usan cuando los registros de la tabla están secuencialmente ordenados por clave de búsqueda del índice.

- Índices dispersos vs densos: Los dispersos requieren menos espacio y sobrecarga de mantenimiento para inserciones y borrados en la tabla. Pero generalmente son más lentos.
- Evaluación del uso de índices: Los índices ofrecen beneficios sustanciales cuando se accede a registros de tablas. Actualizar índices impone una sobrecarga en la modificación de la BD. Cuando un archivo se modifica, cada índice al archivo debe ser actualizado

Índice de multinivel

- Problema: Si el índice primario no cabe en memoria, entonces el acceso pasa a ser costoso.

- Solución: tratar el índice primario mantenido en disco como archivo secuencial y construir un índice disperso en el mismo.

- o Índice externo: índice disperso del índice primario.

- o Índice interno: archivo del índice primario.

- Evaluación: Índices de los dos niveles deben ser actualizados al insertar y borrar en la tabla.

Actualización de índice por borrado de registro: Índice de un solo nivel

o Índice denso: • Si el registro borrado fue el único registro en el archivo con su clave de búsqueda particular: la clave de búsqueda es borrada del índice también. • Sino: o Si la entrada del índice almacena punteros a todos los registros con el mismo valor de clave de búsqueda: se borra el puntero al registro borrado de la entrada del índice. o Sino: si el registro borrado es el primer registro con el valor de la clave de búsqueda, se actualiza la entrada del índice para que apunte al siguiente registro.	o Índice disperso: • Si el índice no contiene una entrada con el valor de clave de búsqueda del registro borrado: nada necesita hacerse en el índice. • Sino: o Si el registro borrado era el único registro con su clave de búsqueda B: • Si el próximo valor de clave de búsqueda tiene una entrada en el índice: la entrada para B es borrada del índice. • Sino: se reemplaza el registro de índice correspondiente con un registro de índice para el próximo valor de clave de búsqueda (en el orden de clave de búsqueda). o Sino: si la entrada del índice para la clave de búsqueda apunta al registro siendo borrado: se actualiza la entrada del índice para apuntar al próximo registro con el mismo valor de clave de búsqueda.
Actualización de índice por inserción de registro: Índice de un solo nivel: Primero se hace búsqueda usando el valor de clave de búsqueda que aparece en el registro a ser insertado.	
o Índice denso: • Si el valor de la clave de búsqueda no aparece en el índice: insertar una entrada de índice con valor de clave de búsqueda en el índice en la posición apropiada. • Sino: o Si la entrada del índice almacena punteros a todos los registros con el mismo valor de clave de búsqueda: el sistema agrega un puntero al nuevo registro en la entrada del índice. o Sino: la entrada del índice almacena un puntero a solo el primer registro con el valor de la clave de búsqueda. Se coloca el registro siendo insertado luego de los otros registros con los mismos valores de clave de búsqueda.	o Índice disperso: • Asumimos que el índice almacena una entrada para cada bloque. • Si el sistema crea un nuevo bloque: inserta el primer valor de la clave de búsqueda (en orden de clave de búsqueda) apareciendo en el nuevo bloque en el índice. • Sino: o Si el nuevo registro tiene el menor valor de clave de búsqueda en su bloque: el sistema actualiza la entrada del índice apuntando al bloque. o Sino: el sistema no hace cambios al índice.

Actualización de índice de varios niveles

- Algoritmos de inserción y borrado para índices de varios niveles son una extensión simple del esquema ya descrito. Luego de inserción o borrado de registro el sistema actualiza el índice de más bajo nivel como se describió.
- Con respecto al segundo nivel, el índice de más bajo nivel es meramente un archivo conteniendo registros. Si hay algún cambio en el índice de más bajo nivel: el sistema actualiza el índice de segundo nivel como se describió.

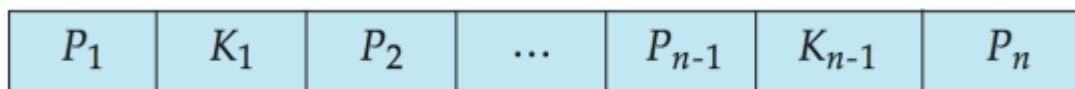
Índices con clave de búsqueda compuesta

- Una clave de búsqueda conteniendo más de un atributo se llama clave de búsqueda compuesta. La estructura del índice es la misma como la de cualquier otro índice; la única diferencia es que la clave de búsqueda del índice contiene una lista de atributos.
- La clave de búsqueda puede representarse como una tupla de valores de la forma: $(a_1, \dots, a_n)$, donde los atributos individuales son: $A_1, \dots, A_n$.
- El orden de los valores de la clave de búsqueda es el orden lexicográfico.

Índices árbol B +

- Los índices de tipo árbol B+ se utilizan para evitar la degradación de rendimiento que ocurre con índices secuenciales cuando el archivo crece, generando muchos bloques de overflow. Mientras que una opción es reorganizar periódicamente el archivo, los índices en forma de árbol B+ ofrecen una solución más eficiente. Estos árboles se mantienen balanceados y reorganizan localmente el índice ante inserciones y borrados, sin necesidad de reorganizar todo el archivo de la tabla. Son ampliamente utilizados por su efectividad en mantener el rendimiento. Satisface las siguientes propiedades:
 - o Todos los caminos desde la raíz a una hoja son de la misma longitud.
 - o Cada nodo que no es la raíz o una hoja tiene entre $n/2$ y n hijos.
 - o Un nodo hoja tiene entre $\lceil (n-1)/2 \rceil$ y $n-1$ valores.
 - o Si la raíz no es una hoja, tiene al menos 2 hijos.
 - o Si la raíz es una hoja, puede tener entre 0 y $n-1$ valores.

- Nodo típico:



- o Los K_i son valores de clave de búsqueda.
- o Los P_i son:
 - Para nodos no hoja: punteros a hijos
 - Para nodos hoja: punteros a registros o buckets de registros.
- o Las claves de búsqueda en un nodo están ordenadas. $K_1 < K_2 < K_3 < \dots < K_{n-1}$
- o Los rangos de valores en cada hoja no se solapan, excepto cuando hay valores de clave de búsqueda duplicados, en cuyo caso un valor puede estar presente en más de una hoja.
- o Asumir inicialmente que no hay claves duplicadas; manejar duplicaciones después.

- Propiedades de un nodo hoja:

- o Para $i = 1, 2, \dots, n-1$, el puntero P_i apunta al registro del archivo con el valor de clave de búsqueda K_i .
- o Si L_i, L_j son nodos hoja y $i < j$, los valores de la clave de búsqueda de L_i son menores o iguales a los valores de la clave de búsqueda de L_j
- o P_n apunta al próximo nodo hoja en el orden de clave de búsqueda.

- Los nodos no hoja forman un índice disperso multinivel en los nodos hoja. Para un nodo no hoja con n punteros:
 - o Todas las claves de búsqueda en el subárbol al cual P_1 apunta son menores que K_1
 - o Para $2 \leq i \leq n-1$: todas las claves de búsqueda en el subárbol al cual P_i apunta tienen valores mayores o iguales a K_{i-1} y menores que K_i .

o Todas las claves de búsqueda en el subárbol al cual P_n apunta tiene valores mayores o iguales que K_{n-1}

Observaciones

- Como las conexiones entre nodos son hechas mediante punteros, los bloques lógicamente cercanos no necesitan estar físicamente cercanos.
- Si hay K valores de clave de búsqueda en la tabla, la altura del árbol B no es mayor a $\lceil \log \lceil n/2 \rceil (K) \rceil$.
- Borrados e inserciones eficientes pq el índice puede ser reestructurado en tiempo logarítmico.

- **Encontrar registro con valor de clave de búsqueda V .**

- *Function find(value V)*

1. $C = \text{root}$
2. While C is not nodo hoja {
 1. Sea i el menor valor tal que $V \leq K_i$.
 2. If no existe: set $C = \text{último puntero non-null en } C$
 3. Else { if ($V = K_i$): $C = P_{i+1}$ else: $C = P_i$ }}
3. Sea i el menor valor tal que $K_i = V$
4. If existe tal valor i : seguir el puntero P_i al registro deseado.
5. Else no existe registro con valor de clave de búsqueda k .

Manejando duplicados

- Con claves de búsqueda duplicadas podemos garantizar que $K_1 \leq K_2 \leq \dots \leq K_n$
- Las claves de búsqueda en el subárbol al cual P_i apunta son K_i , pero no necesariamente $< K_i$
- Entonces, en el nodo padre K_i debe ser igual a V .
- Modificamos el procedimiento de búsqueda como sigue:
 - o Recorrer P_i incluso si $V = K_i$
 - o Tan pronto como alcanzamos un nodo hoja C , chequear si C tiene solo valores de clave de búsqueda menores que V .
- Si es así, sea $C = \text{hermano derecho de } C$ antes de chequear si C contiene V .
- Procedimiento printAll: Usa búsqueda modificada para encontrar primera ocurrencia de V . Recorre las hojas consecutivas para encontrar todas las ocurrencias de V .

Operadores sobre tablas y su implementación

Visión general

- Vamos a pensar una **tabla** como una **lista de tuplas**.
- Un operador del álgebra de tablas es una función que recibe tablas (que cumplen ciertas condiciones) y devuelve una tabla.
- Para procesar una consulta, una sentencia de SQL se puede traducir a una expresión del álgebra de tablas.
- Luego de la traducción, se puede evaluar la consulta del álgebra de tablas.
- Los operadores del álgebra de tablas se llaman operadores lógicos. Estos son definidos usando recursión.
- Un operador físico es un algoritmo específico usado para implementar un operador lógico. Además hacen uso de informaciones, como índices, tamaños de búfer en memoria, etc.

- La evaluación de una expresión del álgebra de tablas va a ser en términos de operadores físicos.
- **Transferencia de bloques de disco:** se refiere al proceso de mover un bloque de datos desde el disco a la memoria principal del sistema, o viceversa.
- **Accesos a bloques:** se refiere al número de veces que el sistema debe leer o escribir bloques en el disco para completar una operación.
- Los costos de todos los algoritmos físicos dependen del tamaño del búfer en memoria principal.

Tablas y esquemas

- Una forma más refinada de dar un esquema que adoptamos es:
 - o $R = (A1::T1, \dots, An::Tn)$
 - o Ti es tipo para valores de Ai
 - o $Dom(R) = T1 \times \dots \times Tn$
- Vieron que $r(A1, \dots, An)$ significa que la tabla r tiene esquema $(A1, \dots, An)$.
- Para dar más detalle ponemos: $r(A1::T1, \dots, An::Tn)$

Operadores

Proyección: $\Pi_{A,C}[] = []$ $\Pi_{A,C}(t:r) = (t.A, t.C) : (\Pi_{A,C} r)$ $= ((\lambda t' \rightarrow (t'.A, t'.C)) t) : (\Pi_{A,C} r)$ $\Pi_{A,C} = \text{map } (\lambda t' \rightarrow (t'.A, t'.C))$	Proyección Generalizada: $\Pi_{f1, \dots, fn} [] = []$ $\Pi_{f1, \dots, fn}(t:r) = (f1(t), \dots, fn(t)) : \Pi_{f1, \dots, fn}(r)$ $= ((\lambda t' \rightarrow (f1(t'), \dots, fn(t'))) t) : \Pi_{f1, \dots, fn}(r)$
• Operador físico: Requiere recorrer todos los registros y realizar una proyección en cada uno. • Se recorren todos los bloques de la tabla. • Estimación de costo = b_r transferencias de bloques + 1 acceso a bloque • b_r = número de bloques conteniendo registros de la tabla r • La proyección generalizada puede ser implementada de la misma manera.	
Selección • Vamos a aplicar la selección a predicados. • Los predicados son funciones booleanas. • Los predicados pueden ser básicos/atómicos o se pueden combinar usando conectivos booleanos $\{\&\&, !!, \text{not}\}$.	• $\sigma_p [] = []$ • $\sigma_p(t:r) = \text{if } p(t) \text{ then } t : (\sigma_p r) \text{ else } \sigma_p r$
• Algoritmo de búsqueda lineal • Escanear cada bloque del archivo y testear todos los registros para ver si satisfacen la condición de selección. • Estimación de costo = b_r transferencias de bloques + 1 acceso a bloque - o b_r denota el número de bloques conteniendo registros de la tabla r	• Algoritmo para índice primario usando árbol B+ con igualdad en clave candidata • Recorrer la altura del árbol más una E/S para recoger el registro. • Cada una de estas operaciones requiere un acceso a bloque y una transferencia de bloque. • El costo es $(h_i + 1)$ transferencias de

	bloque y $(h_i + 1)$ accesos a bloque; h_i denota la altura del índice.
• Algoritmo para índice primario usando árbol B+ igualdad para no clave candidata • un acceso a bloque para cada nivel del árbol y un acceso a bloque para el primer bloque. • Los registros van a estar en bloques consecutivos. • Sea b el número de bloques conteniendo registros con la clave de búsqueda especificada, todos los cuales son leídos. • Estos bloques son bloques hoja asumiendo que están almacenados secuencialmente y no requieren accesos adicionales a bloque. • El costo se compone de: h_i transferencias de bloque y h_i accesos a bloque para recorrer el árbol B+. Donde h_i es la altura del árbol B+. - o Un acceso a bloque para buscar los registros en el archivo de la tabla. - o Un total de b transferencias de bloque para acceder a los registros con la clave de búsqueda.	• Algoritmo para índice secundario usando árbol B+, igualdad en clave candidata • Este caso es similar al índice primario y por lo tanto el costo es el mismo: $(h_i + 1)$ transferencias de bloque y $(h_i + 1)$ accesos a bloque. • donde h_i denota la altura del índice. • Algoritmo para índice secundario usando árbol B+, igualdad en no clave • Aquí el costo de recorrido del índice es el mismo. Sin embargo, cada registro puede estar en un bloque diferente, requiriendo un acceso a bloque por registro. • Sea n el número de registros recogidos. • El costo es $(h_i + n)$ transferencias de bloque y $(h_i + n)$ accesos a bloque. • Esto puede ser muy costoso.
Selecciones involucrando comparaciones	
• Algoritmo para índice primario: • La tabla está ordenada en A. • para $\sigma_{A \geq V}(r)$ usar el índice para encontrar el primer registro $\geq v$ y escanear la tabla secuencialmente desde allí. - o El costo: h_i transferencias de bloque, h_i accesos a bloque y b transferencias de bloque; b es el número de bloques conteniendo registros con $A \geq v$. • para $\sigma_{A \leq V}(r)$ escanear la tabla secuencialmente hasta la primera tupla $> v$.	• Algoritmo para índice secundario: • para $\sigma_{A \geq V}(r)$ usar el índice para encontrar la 1° entrada del índice $\geq v$ y escanear el índice secuencialmente desde allí, para encontrar los punteros a los registros. • Costo = $(h_i + n)$ transferencias de bloque y $(h_i + n)$ accesos a bloque, donde n número de registros con $A \geq v$. • Para $\sigma_{A \leq V}(r)$ escanear hojas del índice encontrando punteros a los registros, hasta la primera entrada $> v$.

• Problema: Para estimar el tamaño de los resultados en cantidad de bloques a escribir a disco para los operadores físicos de selección.

1. Usar una función de probabilidad llamada factor de selectividad para calcular la cantidad de registros del resultado del operador físico de selección. Si el operador de selección o reunión usa predicado P, y el input del operador es r => fs(P, r).

2. A partir de la cantidad de registros, calcular la cantidad de bloques del resultado intermedio. Cantidad de registros resultado intermedio = |r| * fs(P, r)

• Una forma de calcular el factor de selectividad es asumir:

• Uniformidad: todos los valores de un atributo son igualmente probables

• Independencia: condiciones sobre diferentes atributos son independientes

• ¿Cómo calcular el número de bloques si tengo en el resultado N registros de tamaño R cada uno y B es el tamaño del bloque? => NumBloques = $\lceil (N \times R) / B \rceil$.

• Definimos fs(P, r) para distintos tipos de propiedades P y tabla r.
• Desde ahora r tabla, A, A' atributos de r, c y c' constantes.

• **Regla 1:** Asumiendo uniformidad:

• $fs(A = c, r) = 1/V(A, r)$, donde V(A, r) número de distintos valores que aparecen en r para A.

• **Regla 2:** Asumiendo uniformidad, A con valor numérico:

• $fs(A \geq c, r) = (\max(A, r) - c) / (\max(A, r) - \min(A, r))$

• **Regla 3:** Asumiendo uniformidad, A con valor numérico:

• $fs(A < c, r) = (c - \min(A)) / (\max(A) - \min(A) + 1)$

• **Regla 4:** asumiendo uniformidad, A con valor numérico:

• $fs(c \leq A < c', r) = (c' - c) / (\max(A, r) - \min(A, r))$

• **Regla 5:** asumiendo independencia:

• $fs(P_1 \wedge P_2 \wedge \dots \wedge P_n, r) = fs(P_1, r) \cdot fs(P_2, r) \cdot \dots \cdot fs(P_n, r)$

• **Regla 6:** usando propiedad de probabilidades:

• $fs(\neg P, r) = 1 - fs(P, r)$

• **Regla 7:** asumiendo independencia

• $fs(P \vee Q, r) = fs(\neg(\neg P \wedge \neg Q, r)) = 1 - fs(\neg P \wedge \neg Q, r) = 1 - (fs(\neg P, r) * fs(\neg Q, r))$
 $= 1 - ((1 - fs(P, r)) * (1 - fs(Q, r)))$

Producto cartesiano:

• Sean los esquemas:

$r :: (n_1 :: \tau_1, \dots, n_N :: \tau_N) \quad s :: (n'_1 :: \tau'_1, \dots, n'_M :: \tau'_M)$

• Entonces el **esquema del producto cartesiano** es:

$r \times s :: (n_1 :: \tau_1, \dots, n_N :: \tau_N, n'_1 :: \tau'_1, \dots, n'_M :: \tau'_M)$

• Si hay **conflictos de nombres de atributos** se usa el nombre de la tabla para desambiguar.

• Una tabla puede contener **columnas sin nombre**. Para ese caso se usa: $_ : T$

• Voy a poder aplicar el producto cartesiano para tablas donde no todas las columnas tienen nombre.

$[] \times s = []$

$(t:r) \times s = (\text{juntar } t \text{ s}) ++ q$

Recordar que:

$++ : [a] \rightarrow [a] \rightarrow [a]$

$[] ++ l' = l'$

$(x : l) ++ l' = x : (l ++ l')$

• Definimos la **concatenación de tuplas** como la tupla:

$(t; t') = (a_1, \dots, a_N, b_1, \dots, b_M)$

• $\text{juntar } t \text{ []} = []$

• $\text{juntar } t \text{ (u : s)} = (t; u) : (\text{juntar } t \text{ s})$

Ordenamiento

• Especificamos **ordenamiento** usando la idea de insertion sort.

• La función *insert* inserta la tupla *t* en la tabla *r* en la posición que le corresponde: a izquierda las tuplas menores y a derecha las mayores.

$\text{insert}_{a_1, \dots, a_N} t \ r = \sigma_{(a_1, \dots, a_N) < t[a_1, \dots, a_N]}(r) ++ [t] ++ \sigma_{(a_1, \dots, a_N) \geq t[a_1, \dots, a_N]}(r)$

• El **ordenamiento ascendente** se define:

• $O_{a_1, \dots, a_N} ([]) = []$

• $O_{a_1, \dots, a_N} (x:r) = \text{insert}_{a_1, \dots, a_N} (x, O_{a_1, \dots, a_N} (r))$

• El **ordenamiento descendente** se define como la reversa del ordenamiento ascendente.

• $O_{a_1, \dots, a_N}^>(r) = \text{reverse} (O_{a_1, \dots, a_N}(r))$

• $\text{Reverse } [] = []$

• $\text{Reverse } (x : xs) = \text{reverse } xs ++ [x]$

• Cuando queramos ordenar toda la tabla, omitiremos la lista de las columnas.

• Estudiamos el caso de ordenamiento donde las tablas son mayores que la memoria principal. Ordenar tablas que no caben en memoria se llama ordenamiento externo.

El algoritmo de ordenamiento externo más usado: ordenamiento-combinación externo

- Sea M la cantidad de bloques en búfer de memoria principal disponibles para ordenación. 1. Crear corridas ordenadas $i = 0$ repeat Leer M bloques de la tabla en memoria ordenar esos bloques en memoria Escribir los datos ordenados al archivo de ejecución Ri, $i = i + 1$ until el fin de la tabla - Sea N el valor final de i 2. Combinar las corridas	Luego las corridas son combinadas. Suponemos que $N < M$. Leer un bloque de cada uno de los N archivos Ri en un bloque del búfer de memoria de corrida Ri. repeat Seleccionar la primera tupla en el orden de ordenamiento entre todos los bloques del búfer. Escribir la tupla al búfer de salida y borrarla del bloque del búfer donde estaba. if búfer de salida está lleno then escribirlo a disco, borrar contenido de búfer de salida. if búfer de corrida Ri está vacío y no fin de archivo Ri then leer el próximo bloque de la corrida Ri en el bloque de búfer de memoria correspondiente a esa corrida. until todos los bloques de búfer de las corridas estén vacíos.
- Si $N \geq M$, varios ciclos de combinación son requeridos. - Se dividen las corridas en grupos contiguos de $M-1$ corridas. - Cada grupo de $M-1$ corridas es combinado (usando el algoritmo anterior), dando lugar a una nueva corrida. - Hacer los dos pasos anteriores constituye una pasada. - La misma reduce el número de corridas por un factor de $M-1$ y crea corridas más largas por el mismo factor. - Si el número de corridas resultantes de esa pasada es mayor a M, - se realiza otra pasada de la misma manera, pero con las corridas creadas en la primera pasada. - Observar que cada pasada reduce el número de secuencias en $M-1$. - Se repiten las pasadas hasta que número de corridas generadas sea menor que M. - Luego el último ciclo de combinación genera el resultado ordenado.	Costo del ordenamiento: - Leer justificación en el libro de Silberschatz. - Número de transferencias de bloques para ordenamiento externo: $b_r \cdot 2 \lceil \log_{M-1} (b_r / M) \rceil + 1$ - Donde b_r es cantidad de bloques de la tabla - Cantidad de accesos a bloque: $2 \lceil b_r / M \rceil + \lceil b_r / b_b \rceil (2 \lceil \log_{M/bb-1} (b_r / M) \rceil - 1)$, - donde b_b bloques son alojados en cada corrida

Reunión selectiva - Sean dos tablas de esquema: $r(n_1 :: \tau_1, \dots, n_N :: \tau_N)$ y $s(m_1 :: \tau'_1, \dots, m_M :: \tau'_M)$ - Entonces: $r_{a_1, \dots, a_i} \bowtie b_1, \dots, b_i \text{ } s :: (n_1 :: \tau_1, \dots, n_N :: \tau_N, c_1, \dots, c_{M-i})$ para $j \in [1, M-i]$, $c_j \in \{m_1, \dots, m_M\} \setminus \{b_1, \dots, b_i\}$.	- La reunión selectiva se puede definir usando los operadores vistos: $r_{a_1, \dots, a_i} \bowtie b_1, \dots, b_i \text{ } s = \prod_{n_1, \dots, n_N, c_1, \dots, c_{M-i}} (\sigma_{a_1=b_1 \wedge \dots \wedge a_i=b_i} (r \times s))$ - Un tipo de reunión selectiva que se usa mucho es reunión natural: se aplica reunión selectiva a todos los atributos con el mismo nombre en las dos tablas. - Sean $r(n_1 :: \tau_1, \dots, n_N :: \tau_N)$ y $s(m_1 :: \tau'_1, \dots, m_M :: \tau'_M)$, definimos la reunión natural de r y s como: $r \bowtie s = r_{a_1, \dots, a_i} \bowtie a_1, \dots, a_i \text{ } s$ donde $\{a_1, \dots, a_i\} = \{n_1, \dots, n_N\} \cap \{m_1, \dots, m_M\}$.
Algoritmo de reunión selectiva de loop anidado for each tuple t_r in r do begin for each tuple t_s in s do begin test pair (t_r, t_s) to see if they satisfy the join condition if they do, add $t_r \bullet t_s$ to the result. end end	- Nomenclatura: r se llama tabla externa y s se llama tabla interna de la reunión selectiva. - El algoritmo anterior no requiere de índices. - El algoritmo anterior es costoso porque examina todo par de tuplas en las dos tablas. - En el peor caso, si hay suficiente memoria solo para sostener un bloque para cada tabla, el costo estimado es: $n_r \cdot b_s + b_r$ transferencias de bloques $n_r + b_r$ accesos a bloques. - donde n_r es cantidad de registros en r, b_r es la cantidad de bloques en r, y b_s es la cantidad de bloques en s.
Algoritmo de reunión selectiva de loop anidado de bloques for each block B_r of r do begin for each block B_s of s do begin for each tuple t_r in B_r do begin for each tuple t_s in B_s do begin if (t_r, t_s) satisface condición de reunión selectiva then add $t_r \bullet t_s$ to the result. end end end end	Estimación de peor caso: $b_r \cdot b_s + b_r$ transferencias de bloque + $2 \cdot b_r$ accesos a bloque - Cada bloque en la tabla interna s es leído una sola vez por cada bloque en la tabla externa. Mejora del algoritmo anterior: M tamaño de memoria en bloques. - Usar $M-2$ bloques de disco para tabla externa, usar los restantes 2 bloques para tabla interna y salida. - Costo = $\lceil b_r / (M-2) \rceil \cdot b_s + b_r$ transferencias de bloques + $2 \lceil b_r / (M-2) \rceil$ accesos a bloque
Algoritmo de reunión selectiva de loop anidado indexado - Asumir que hay un índice en el atributo de la relación interna de la reunión. - Para cada tupla t_r en la tabla externa r usar el índice para buscar tuplas en s que satisfacen la condición de reunión con tupla t_r. Peor caso: el búfer tiene espacio para solo un bloque de r y un bloque del índice y para cada tupla en r hacemos búsqueda en índice de s. - Para leer r en el peor caso hay b_r accesos a bloque y b_r transferencias de bloque. - A esto hay que sumarle $n_r \cdot c$, n_r es cantidad de registros en r. - Aquí c es el costo de recorrer el índice y recolectar todas las tuplas de s que cazan para una tupla de r. - El valor c puede estimarse como el costo de una selección en s usando la condición de la reunión.	Algoritmo de reunión por mezcla: - Supongamos que hay un atributo de la reunión. - Ordenar ambas tablas en el atributo de la reunión. - Aplicar la reunión selectiva a las tablas ordenadas. - Si las tablas consideradas son r y s y se quiere hacer reunión selectiva de r y s: - Se trabaja con variables para tupla actual de r y tupla actual de s - El algoritmo se basa en aplicar las siguientes dos ideas: - Si tupla actual t de r caza con tupla actual t' de s: se agrega $t \bowtie t'$ al resultado. - Se repite esto hasta que haya tupla de s que no caza con t o se acabó de recorrer s. - Si tupla actual de r no caza con tupla actual de s, se pasa a siguiente tupla de r - Se repite esto hasta que se acabe de recorrer r o se encuentra tupla de r que caza con tupla actual de s.

• Análisis del algoritmo de reunión por mezcla: • Cada bloque necesita ser leído una sola vez asumiendo que todas las tuplas para un valor dado de los atributos del join entran en memoria. • Entonces el costo de reunión por mezcla es: • $b_r + b_s$ transferencia de bloques $+ \lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil$ accesos a bloque • Donde b_b bloques de búfer son asignados a cada tabla. • A esto se suma el costo de ordenar las tablas si no estaban ordenadas.	• Algoritmo de reunión por mezcla híbrido: • Suposición: la primera tabla está ordenada y la segunda tabla está desordenada, pero tiene un índice secundario árbol B+ en los atributos de la reunión. • Algoritmo: 1. El algoritmo combina la tabla ordenada con las entradas hoja del índice secundario B+. • El archivo resultante contiene tuplas de la relación ordenada y direcciones para las tuplas de la relación no ordenada. 2. Se ordena el resultado por las direcciones de las tuplas de la relación no ordenada. 3. Permite el retorno eficiente de las tuplas correspondientes en el orden de almacenamiento físico para terminar la reunión. • Costo del segundo paso: Costo de ordenar el archivo resultante. • Costo del tercer paso: costo de acceso a tuplas en el orden de almacenamiento físico.
• Problema: Hay que estimar el tamaño de los resultados en cantidad de bloques a escribir a disco para los operadores físicos de reunión (selectiva y natural). • Solución: 1. Usar una función de probabilidad llamada factor de selectividad para calcular la cantidad de registros del resultado del operador físico de reunión. 2. A partir de cantidad de registros, calcular la cantidad de bloques del resultado intermedio. • La cantidad de registros para el resultado de la reunión viene dada por $r * s * fs(P, r, s)$	• Si r tabla y A atributo de r, entonces $V(A, r)$ número de distintos valores que aparecen en r para A. • Regla: asumiendo uniformidad: • $fs(r.A = s.B, r, s) = 1 / \max(V(A, r), V(B, s))$ • la uniformidad significa: para cada valor de atributo A/B en una tabla hay la misma cantidad de tuplas por el atributo B/A que cazan en la otra tabla. • Observaciones: • Si A clave candidata de r: $fs(r.A = s.B, r, s) = 1 / \max(r , V(B, s))$ • Si B clave foránea en s referenciando r: $fs(r.A = s.B, r, s) = 1 / r$ • Si toda tupla en r produce tuplas en la reunión selectiva: • $fs(r.A = s.B, r, s) = 1 / V(B, s)$ • Si se puede asumir independencia: • $fs(r.A = s.B \wedge r.A' = s.B', r, s) = fs(r.A = s.B, r, s) * fs(r.A' = s.B', r, s)$

Remover duplicados • La operación de remoción de duplicados se define recursivamente así: 1. $V[] = []$ 2. $V(t:r) = \text{if } t \in r \text{ then } V(r) \text{ else } V(r)$ • El predicado en la segunda ecuación se define del siguiente modo: • $t \in [] = \text{false}$ • $t \in (u:r) = t == u \vee t \in r$	Operador físico de remoción de duplicados • Implementación de remoción de duplicados ○ Se ordena la tabla; al hacerlo, todos los duplicados van a ser adyacentes entre sí. Y todos los duplicados menos uno, pueden ser eliminados. ○ Optimización: en ordenación externa duplicados pueden ser eliminados durante generación de corridas, así como en pasos de combinación intermedios. ○ El peor caso de costo para eliminación de duplicados es el mismo que el peor caso de costo para ordenar una tabla.
--	---

Concatenación de tablas • ¿Qué operación de tablas corresponde a la unión de conjuntos? • Una posibilidad es la concatenación de tablas. • Es como la concatenación de listas solo que ahora el resultado tiene un esquema. • Los esquemas de los operandos deben ser compatibles (i.e. igual cantidad de columnas y mismos tipos en las mismas posiciones). Sean $r(n_1 :: \tau_1, \dots, n_N :: \tau_N)$ y $s(m_1 :: \tau_1, \dots, m_M :: \tau_N)$ $r++s :: (n_1 :: \tau_1, \dots, n_N :: \tau_N)$ • $[] ++ s = s$ • $(t:r)++s = t: (r++s)$	• Concatenación requiere leer ambas tablas: primero la de la izquierda y luego la de la derecha. • A medida que se lee una tabla se genera el resultado. • En el peor caso para computar $r++s$ vamos a necesitar: • $b_r + b_s$ transferencias de bloques y accesos a bloques • Si el resultado es un resultado intermedio: se escriben $b_r + b_s$ bloques en disco. Este costo tmb va
--	--

Resta Los esquemas de las tablas deben ser compatibles $r(n_1 :: \tau_1, \dots, n_N :: \tau_N) \text{ y } s(m_1 :: \tau_1, \dots, m_M :: \tau_N)$ El esquema de la resta es el del primer operando. $r \setminus s :: (n_1 :: \tau_1, \dots, n_N :: \tau_N)$ $r \setminus s = \sigma_{(t \rightarrow t \notin s)}(r)$	• Algoritmo: • Para calcular la resta de dos tablas, podemos primero ordenarlas a ambas por la clave primaria. • Luego podemos escanear una vez cada tabla ordenada para producir el resultado. • Costo: • Suponiendo las tablas r, s ordenadas de ese modo. • Si un solo bloque en búfer se usa por tabla: • $b_r + b_s$ transferencias de bloques • $b_r + b_s$ accesos a bloque
--	---

Intersección - Los esquemas de las tablas deben ser compatibles $r(n_1 :: \tau_1, \dots, n_N :: \tau_N) \text{ y } s(m_1 :: \tau_1, \dots, m_M :: \tau_N)$ - El esquema de la intersección es el del primer operando. $r \cap s :: (n_1 :: \tau_1, \dots, n_N :: \tau_N)$ $r \cap s = \sigma_{(t \rightarrow t \in s)}(r)$	Algoritmo: - Para calcular la intersección de dos tablas, podemos primero ordenarlas a ambas por la clave primaria. - Luego podemos escanear una vez cada tabla ordenada para producir el resultado. Costo: Suponiendo las tablas r, s ordenadas de ese modo, $r \cap s$ requiere: - Si un solo bloque en búfer se usa por tabla: $b_r + b_s$: transferencias de bloques $b_r + b_s$: accesos a bloque - El número de accesos a bloque puede ser reducido alojando bloques extra en búfer.
---	--

Procesamiento de consultas

- Para procesar una consulta, una consulta de SQL se puede traducir a una expresión del álgebra de tablas. Luego se puede evaluar la consulta del álgebra de tablas, esta evaluación va a estar en términos de operadores físicos.
- Una expresión de álgebra de tablas puede tener varias expresiones equivalentes.
- Dada una consulta C del álgebra de tablas un **plan de evaluación** consiste de una expresión E equivalente a C y operadores físicos para los operadores lógicos de E .
- La máquina de ejecución de consultas toma el plan de evaluación de consulta, ejecuta ese plan y retorna las respuestas de la consulta.
- Los diferentes planes de evaluación para una consulta dada pueden tener diferentes costos. El SGBD debería construir un plan de evaluación que minimiza el costo de evaluación de consultas.
- El árbol binario de ejecución de una consulta es el árbol binario de una expresión de consulta. Donde los nodos hoja son tablas de la base de datos y los nodos internos son operadores del álgebra de tablas
- La evaluación de un árbol binario de ejecución va a estar en términos de operadores físicos.
- El resultado de evaluar un operador de un nodo interno del árbol binario de ejecución que no es la raíz del árbol se llama resultado intermedio.
- Para la evaluación del árbol binario de ejecución hay dos enfoques:
 - Materialización:** los resultados intermedios se guardan en disco en tablas temporales a las cuales tiene acceso el sistema gestor de BD (SGBD). No hay índices sobre este tipo de tablas.
 - Encausamiento:** a medida que se van generando los resultados intermedios se van pasando al siguiente operador. Los resultados intermedios no se guardan en disco.
- Materialización:**
 - Cambiar nodos lógicos por físicos en el árbol binario de ejecución.
 - Si hay más de un operador físico posible, elegir el menos costoso.
 - Evaluamos un operador físico por vez comenzando en el nivel más bajo.
 - Usamos resultados intermedios en tablas temporales para evaluar los operadores físicos del siguiente nivel.
- Repaso: para estimar la cantidad de registros del resultado intermedio para los operadores de selección y reunión selectiva se usa el factor de selectividad.
 - Para selección: cantidad de registros resultado intermedio = $|r| * fs(P, r)$
 - Para reunión: Cantidad de registros resultado intermedio = $|r| * |s| * fs(P, r, s)$
 - Resultado intermedio N registros de tamaño R cada uno y B es el tamaño del bloque, entonces la cantidad de bloques del resultado intermedio $\Rightarrow$ NumBloques = $\lceil (N \times R) / B \rceil$.
- Fase 1: decidir el plan de ejecución
 - Armar el árbol binario de ejecución
 - Calcular el factor de selectividad para selecciones y reuniones
 - Decidir operadores físicos.
- Solo se usan índices si la tabla de la BD lo amerita.

- Fase 2: estimar el costo de ejecutar el plan de evaluación
 1. Calcular el tamaño en bloques de las tablas de la BD
 2. Calcular el tamaño de los resultados intermedios en bloques
 3. Calcular el costo de los operadores físicos
 4. Sumar los costos totales

Introducción a la optimización de consultas

- Pasos en optimización de consultas basada en costo:
 1. Generar expresiones lógicamente equivalentes usando reglas de equivalencia.
 2. Anotar expresiones resultantes para obtener planes de consulta alternativos.
 3. Elegir el plan más económico basado en el costo estimado.
- El costo estimado de un plan se basa en: Información estadística acerca de tablas, estimación estadística para resultados intermedios.
- Para computar el costo de expresiones complejas: Fórmula de costo para algoritmos, computados usando estadísticas.
- Las estadísticas son computadas periódicamente porque tienden a no cambiar radicalmente en un corto tiempo; estadísticas algo imprecisas son útiles siempre que sean aplicadas consistentemente a todos los planes.
- El número de transferencias de bloques es influenciado por:
 1. Los operadores lógicos elegidos para implementar la consulta.
 2. El tamaño de los resultados intermedios.
 3. Los operadores físicos usados para implementar los operadores lógicos.
 4. El método para pasar argumentos de un operador físico al siguiente.
 5. El orden de operaciones similares, especialmente reuniones y selecciones.
- Dos expresiones del álgebra de tablas son equivalentes por igualdad si las dos tablas son equivalentes en esquema e información.
- Dos expresiones del álgebra de tablas son equivalentes módulo ordenamiento de registros si las dos expresiones generan el mismo multiconjunto de tuplas.
- Una regla de equivalencia dice que las expresiones de dos formas son equivalentes usando uno de los tipos de equivalencia de arriba.

Reglas de equivalencia 1. La selección conjuntiva de operaciones puede ser deconstruida en una secuencia de selecciones individuales: $S_{q_1 \wedge q_2}(E) = S_{q_1}(S_{q_2}(E))$ 2. Las operaciones de selección son conmutativas: $S_{q_1}(S_{q_2}(E)) = S_{q_2}(S_{q_1}(E))$ 3. Solo la última en una secuencia de operaciones proyección es necesitada, las otras pueden ser omitidas. $\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_n}(E)$	• Nomenclatura: • Usaremos Θ_r^N para indicar una lista de N columnas de la tabla r. • Usaremos $\Theta_r^N = \Theta_s^N$ para indicar la igualación de cada columna de Θ_r^N con las columnas de Θ_s^N. Omitiremos la tabla y/o cantidad de columnas si no hay ambigüedad. • Indicaremos con p_r a una proposición que sólo utilice columnas de r.
4. La reunión natural es asociativa: $(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$ 5. La siguiente regla es parecida a la definición de reunión selectiva: $\Pi_{\theta}(\sigma_{\theta_r^N = \theta_s^N}(r \times s)) = r \bowtie_{\theta_r^N \bowtie \theta_s^N} s \quad \text{con } \theta \text{ las columnas de } \bowtie$ 6. La siguiente regla muestra cómo podemos expresar una selección de una reunión selectiva por medio de una reunión selectiva. $\sigma_{\theta_r^N = \theta_s^N}(r \bowtie_{\theta_r^M \bowtie \theta_s^M} s) = r \bowtie_{\theta_r^N, \theta_r^M \bowtie \theta_s^N, \theta_s^M} s$	7. La siguiente regla nos permite aplicar selección antes de reunión selectiva: $\sigma_{p_r}(r \bowtie_{\theta_r^N \bowtie \theta_s^N} s) = \sigma_{p_r}(r) \bowtie_{\theta_r^N \bowtie \theta_s^N} s$ 8. La siguiente regla se deduce de reglas previas: $\sigma_{p_r \wedge p_s}(r \bowtie_{\theta_r^N \bowtie \theta_s^N} s) = \sigma_{p_r}(r) \bowtie_{\theta_r^N \bowtie \theta_s^N} \sigma_{p_s}(s)$ 9. La siguiente regla muestra cómo se comporta la proyección cuando se usa junto con la reunión selectiva. $\Pi_{\theta_r^N, \theta_s^O}(r \bowtie_{\theta_r^M \bowtie \theta_s^M} s) = \Pi_{\theta_r^N}(r) \bowtie_{\theta_r^M \bowtie \theta_s^M} \Pi_{\theta_s^O}(s) \quad \text{con } \theta_r^M \subseteq \theta_r^N \text{ y } \theta_s^M \subseteq \theta_s^N$

10. La concatenación "conmuta" alterando solo el orden de las tuplas: $r \text{ ++ } s =_o s \text{ ++ } r$ 11. La concatenación es asociativa. 12. Selección distribuye con concatenación, intersección y resta $\sigma_p(r \star s) = \sigma_p(r) \star \sigma_p(s) \quad \text{con } \star \in \{++, \cap, \setminus\}$ 13. La siguiente propiedad es más débil que la anterior. $\sigma_p(r \star s) = \sigma_p(r) \star s \quad \text{con } \star \in \{\cap, \setminus\}$ 14. La proyección distribuye con la concatenación: $\Pi_\theta(r \text{ ++ } s) = \Pi_\theta(r) \text{ ++ } \Pi_\theta(s)$	15. Eliminación de duplicados distribuye con reunión natural: $v(r \bowtie s) = v(r) \bowtie v(s)$ 16. Eliminación de duplicados conmuta con selección: $v(\sigma_r(r)) = \sigma_r(v(r))$
---	---

Optimización Heurística

- Problema: Optimización basada en costo es cara.
- Una heurística es un enfoque para resolución de problemas que usa un método práctico o varios atajos con el fin de producir soluciones que pueden no ser óptimas, pero son suficientes dado el tiempo o recursos limitados.
- Solución: Los sistemas pueden usar heurísticas para reducir el número de elecciones que deben ser hechas cuando se trabaja con costos.
 - o La optimización heurística transforma el árbol de consulta usando un conjunto de reglas que típicamente pero no siempre mejoran el desempeño de ejecución.
- También una optimización heurística puede referirse a operadores físicos a usar.
- Ejemplo de optimización heurística:
 - o Realizar selección tempranamente (reduce el número de tuplas)
 - o Realizar proyección temprano (reduce el número de atributos)
 - o Hacer la selección más restrictiva (i.e. con tamaño de resultado menor) antes de hacer otras selecciones.

Selección de orden de reunión basado en costo

- Para la selección de orden de reunión natural basado en costo hay algoritmos de programación dinámica.
- En lugar de generar todas las expresiones de reunión posibles, se consideran conjuntos de relaciones a reunir y optimizar.
- Estos conjuntos se guardan en tablas.
- Estas tablas tienen muchos menos elementos que la cantidad de reuniones posibles.
- Usando programación dinámica el orden de reunión de menor costo para cada subconjunto de $\{r_1, \dots, r_n\}$ es computado solo una vez y almacenado para uso futuro.